

**BACHELOR 3 – Semestre 2 (2025-26)****BDD AVANCÉE – PROJET FINAL****Date de lancement : à préciser****Enseignant : ROCHE Milo****Durée totale : ~10 heures (réparties sur plusieurs séances + travail personnel)****Modalité : projet en groupe de 4**

Documents et outils autorisés :

- Cours et supports de cours : OUI
- Internet, documentation officielle des frameworks/SGBD : OUI
- Outils d'IA générative (avec déclaration obligatoire) : OUI sous conditions
- Travail collaboratif au sein du groupe : OBLIGATOIRE

**⚠ Avertissement orthographe et lisibilité**

Le correcteur pourra retirer jusqu'à 2 points si le dossier n'est pas lisible et/ou comporte des fautes d'orthographe répétées. La qualité rédactionnelle, la cohérence visuelle du dossier et la propreté du dépôt Git font partie intégrante de l'évaluation.

**SUJET : APPLICATION WEB POLYGLOTTE — DEV + SQL + NoSQL**

**Vous êtes en équipe de quatre personnes.** Vous devez concevoir, modéliser, développer, conteneuriser, documenter et soutenir oralement une application web complète, de A à Z, en exploitant simultanément deux bases de données de natures différentes : une base relationnelle (SQL) et une base NoSQL.

**Le sujet fonctionnel est libre :** vous choisissez le domaine métier qui vous parle (gestion d'événements, plateforme communautaire, marketplace, application de quantified-self, jeu en ligne, outil pédagogique, réseau social de niche, suivi sportif, plateforme musicale, gestion de bibliothèque, application immobilière, agrégateur d'actualités, etc.). Le seul impératif est que votre choix justifie naturellement l'utilisation conjointe d'une BDD relationnelle et d'une BDD NoSQL — c'est-à-dire qu'il existe dans votre métier au moins une famille de données qui se prête mieux à un modèle non relationnel (documents flexibles, données massives en lecture, graphes de relations, séries temporelles, sessions/cache, etc.).

**Vous êtes libres** des technologies (langages, frameworks back et front, ORM/ODM, bibliothèques), de l'identité visuelle et de l'organisation interne du groupe. Vous êtes **fortement encouragés** à utiliser Docker (Dockerfile + docker-compose) pour packager toute la stack.

## Consignes générales

- Vous travaillez en groupe de 3 ou 4 personnes; les groupes de 5 sont refusés.
- Le barème est sur 20 points. La BDD (SQL + NoSQL) pèse à elle seule beaucoup c'est le cœur de l'évaluation.
- Vous disposez d'environ 10 heures encadrées en salle, complétées par le travail personnel nécessaire à la livraison. Aucun délai supplémentaire ne sera accordé.
- Toutes les productions (code, dossier, scripts SQL, schémas) doivent être originales et personnelles à votre groupe. Le plagiat entre groupes ou depuis Internet est sanctionné par un zéro éliminatoire.
- L'usage de l'IA générative est autorisé sous condition de déclaration explicite : une section dédiée « Usage de l'IA » doit figurer dans votre dossier, voire sur chaque partie (modèle utilisé, prompts représentatifs, parties produites avec, parties reformulées, parts d'auto-rédaction). Une copie générée à 100% par IA, sans recul critique, sera plafonnée.
- Toutes les communications du groupe (board de gestion de projet, dépôt Git, échanges de fichiers) doivent être traçables. Pas de fichier flottant sur Discord.
- La lisibilité, l'orthographe, la cohérence visuelle et la qualité rédactionnelle du dossier sont prises en compte (jusqu'à -2 points).

## UTILISATION DE L'IA GÉNÉRATIVE

---

L'utilisation d'outils d'IA générative (ChatGPT, Claude, Copilot, Gemini, etc.) est **autorisée** pendant cette épreuve. Cependant, elle doit être **systématiquement transparente et justifiée**. La transparence sur l'usage de l'IA fait partie intégrante de la notation.

### Règles à respecter impérativement :

- Si vous utilisez l'IA pour vous aider (débuguer, générer du code, rappeler un concept, rédiger un paragraphe, etc.), vous devez le **signaler explicitement** dans votre rapport à l'endroit concerné.
- Pour chaque utilisation, précisez : le **fournisseur et le modèle** utilisé (ex : ChatGPT GPT-4o, Claude Sonnet, GitHub Copilot...), le **prompt exact** que vous avez soumis, et la **réponse obtenue** (ou un résumé si elle est longue).
- Expliquez **pourquoi** vous avez choisi d'utiliser l'IA plutôt qu'une recherche classique (documentation, Stack Overflow, cours). Exemples de justifications acceptées : gain de temps sur un problème bloquant, reformulation d'un concept mal compris, génération d'un squelette de code à adapter, etc.
- Indiquez si vous avez **modifié ou adapté** la réponse de l'IA, et si oui, en quoi et pourquoi.

### Ce qui sera valorisé :

- Une utilisation pertinente et ciblée de l'IA (pas du copier-coller intégral sans recul)
- La transparence totale : mieux vaut déclarer une utilisation que de la masquer
- La capacité à expliquer et critiquer la réponse de l'IA (montrer que vous comprenez)

### Ce qui sera pénalisé :

- L'utilisation non déclarée d'IA détectée lors de la correction
- Le copier-coller brut d'une réponse IA sans adaptation, sans explication et sans recul critique
- L'incapacité à expliquer ou défendre un élément de votre rendu produit par IA

*Conseil : vous pouvez dédier une sous-section « Utilisation de l'IA » à la fin de chaque partie, ou annoter directement chaque passage concerné. L'important est que le correcteur puisse identifier clairement ce qui vient de vous et ce qui vient d'un outil.*

## Contexte pédagogique

**Tout au long de vos années, vous avez vu :** la conception relationnelle (MERISE, MCD, MLD, normalisation, cardinalités, dépendances fonctionnelles), le SQL avancé (jointures multiples, sous-requêtes, vues, déclencheurs, procédures stockées, transactions, indexation, optimisation), la modélisation NoSQL (documents, clé-valeur, colonnes larges, graphes), MongoDB en pratique, le théorème CAP, les compromis ACID/BASE, et les architectures polyglotte.

**Ce projet final est l'occasion de mobiliser** l'ensemble de ces acquis sur un cas réel et complet. Il ne s'agit plus d'exercices isolés sur une base d'exemple : il s'agit de défendre des choix d'architecture, de modélisation et d'implémentation, devant un jury, en argumentant techniquement chaque décision.

### Ce qui sera valorisé

- Une justification rigoureuse du choix du type de NoSQL retenu (documentaire, clé-valeur, colonnes, graphe) avec argumentation contre les autres types.
- Un MCD propre, normalisé jusqu'à la 3NF a minima, avec gestion réfléchie des héritages, agrégations et associations n-aires.
- L'utilisation effective des objets SQL avancés vus en cours : au moins une vue, un trigger, une procédure stockée, une transaction explicite, un index non trivial.
- Une exploitation réelle des spécificités du NoSQL retenu : pour MongoDB par exemple, l'utilisation du framework d'agrégation, des index TTL ou texte, du sharding s'il est pertinent.
- Une réflexion sur la cohérence inter-bases (données qui transitent entre SQL et NoSQL : qui est source de vérité ? quelle stratégie de synchronisation ?).

### Ce qui sera sanctionné

- Une BDD NoSQL purement décorative (par exemple : une seule collection MongoDB qui stocke des logs, sans agrégation, sans justification métier réelle).
- Un MCD ou Diagramme de Classes avec des erreurs de cardinalités, des associations non réduites, ou des entités fusionnables non fusionnées.
- Des requêtes SQL toutes faites à coups de SELECT \* sans aucune jointure ou agrégation.
- L'absence de scripts SQL d'initialisation reproductibles (création de schéma + jeu de données de démo).
- Un dossier de conception qui ne suit pas le sommaire imposé.

## Partie 1 : Cahier des charges technique (obligations minimales)

### 1.1 Architecture applicative

- Application web complète : un back-end exposant une API et un front-end consommant cette API. Le rendu purement back-end (templates serveur uniquement) est refusé.
- Séparation claire back/front, idéalement deux dépôts ou deux dossiers distincts dans un mono-repo.
- Toute la stack doit être démarrable en une commande (idéalement docker-compose up). Le correcteur doit pouvoir cloner, lancer et utiliser l'application sans plus de 5 minutes de configuration.
- Si vous pouvez l'héberger, c'est encore mieux :)

### 1.2 API : 16 endpoints REST minimum OU GraphQL équivalent

*Vous avez le choix entre deux conventions d'API.*

#### Option A — API REST (16 endpoints minimum)

- Couverture CRUD complète sur **au minimum 4 ressources métier distinctes** (soit 4 × CRUD ≈ 16 endpoints incompressibles).
- Au moins 2 endpoints d'authentification/autorisation (login, refresh, logout, me, register...).
- Au moins 2 endpoints transverses non-CRUD : recherche full-text, agrégation/statistiques, export, action métier (« valider une commande », « publier un post », « rejoindre un groupe »...).
- Respect des codes HTTP standards (200, 201, 204, 400, 401, 403, 404, 409, 422, 500). Une API qui renvoie 200 partout est sanctionnée.
- Pagination, filtrage et tri sur les endpoints de listing.
- Une documentation auto-générée (Swagger/OpenAPI, Scalar, Redoc) accessible via une URL.

#### Option B — API GraphQL (richesse équivalente)

- Schéma comportant au minimum : 4 types métier principaux, 8 queries, 8 mutations, 1 subscription si pertinent.
- Résolveurs avec gestion de la profondeur, du N+1 (DataLoader ou équivalent), et de l'authentification par directive ou middleware.
- Un explorateur (GraphQL Playground, Apollo Studio) accessible en local.
- Documentation du schéma (descriptions sur chaque type/champ).

### 1.3 Bases de données : une SQL + une NoSQL (obligatoire)

#### Règle fondamentale du projet

Vous devez utiliser **SIMULTANÉMENT**, dans la même application, une base de données relationnelle (SQL) ET une base de données NoSQL. Les deux doivent être réellement exploitées par votre back-end et avoir un rôle métier identifiable. Une BDD purement « pour faire joli » est sanctionnée.

### Côté SQL (choix libre du SGBD)

- SGBD au choix : MySQL, MariaDB, PostgreSQL. Justifiez votre choix dans le dossier.
- Schéma normalisé jusqu'à la 3NF a minima.
- Utilisation effective et justifiée d'au moins : 2 vues, 2 triggers, 3 procédures stockées, 1 transaction explicite (BEGIN/COMMIT/ROLLBACK), 1 index ajouté manuellement (autre que la PK).
- Au moins une requête métier complexe avec : 4 jointures minimum, 2 agrégation (GROUP BY + HAVING), 2 sous-requêtes corrélées OU 1 CTE.
- Script d'initialisation reproductible (schéma + données de démo) versionné.

### Côté NoSQL

- **Type de NoSQL au choix** parmi : documentaire (MongoDB, CouchDB), clé-valeur (Redis, DynamoDB), colonnes (Cassandra, ScyllaDB), graphe (Neo4j, ArangoDB).
- **Justification obligatoire du type retenu** dans le dossier : pourquoi ce type est le mieux adapté à la nature des données stockées, et pourquoi les trois autres types sont moins pertinents pour ce cas précis.
- Exploitation effective des spécificités du moteur retenu (pas seulement get/set) : agrégations MongoDB, TTL, index full-text ou géospatial, pub/sub Redis, traversées de graphe, requêtes Cypher, etc.
- Au moins une opération métier de votre application doit reposer principalement sur le NoSQL.

## 1.4 Conteneurisation (fortement recommandée)

- Dockerfile pour le back-end (multi-stage si compilé).
- Dockerfile pour le front-end (build + serveur statique).
- docker-compose.yml orchestrant : back, front, SGBD SQL, SGBD NoSQL (et tout service additionnel pertinent : reverse-proxy, cache, etc.).
- Aucun secret en dur dans les images : utilisation de variables d'environnement et d'un fichier .env.example fourni.
- Volumes nommés pour la persistance des données.
- Bonus : restart politics, healthchecks, dépendances depends\_on avec condition: service\_healthy.

## 1.5 Charte graphique et UX

- Une identité visuelle propre conçue par le groupe : nom du produit, logo (même simple), palette de couleurs cohérente (3 à 5 couleurs maximum, code hexa explicite), typographie (1 à 2 polices), iconographie.
- La charte est documentée dans le dossier de conception (page dédiée avec extraits de la palette et exemples d'application).
- Front responsive (mobile + desktop a minima) avec preuve dans le dossier et pendant la soutenance finale.
- **Interdit** : utiliser tel quel un template Bootstrap par défaut ou un kit Tailwind « starter » sans personnalisation. Le rendu doit être identifiable comme étant le vôtre.

## Partie 2 : Gestion de projet et organisation

L'organisation du groupe est un livrable à part entière. Vous devez démontrer que votre équipe a fonctionné comme une équipe technique réelle, pas comme quatre personnes qui codent chacune dans leur coin.

### 2.1 Méthode imposée : Agile (Scrum allégé ou Kanban)

- Outil de gestion de projet au choix : Trello, Notion, GitLab Issues, GitHub Projects, Jira, Linear. Le board doit être accessible (lien dans le dossier ou screenshots datés).
- Colonnes minimum imposées : Backlog / À faire / En cours / À valider / Problématique / Terminé.
- Chaque ticket doit avoir : un titre, une description, un assigné, une estimation de complexité (T-shirt sizing ou points), des labels (bug/feature/techdette/doc).
- Si Scrum : 2 sprints minimum, avec planning, daily court (10/15 min), review et rétro documentés.
- Si Kanban : limite de WIP affichée, lead time mesuré, au moins une rétrospective intermédiaire documentée.

### 2.2 Versionnement Git

- Dépôt Git public ou accessible (GitHub, GitLab, Bitbucket). Pas de dépôt local non poussé.
- Stratégie de branching documentée et appliquée : minimum branche main protégée + branches de feature.
- Convention de nommage des commits (Conventional Commits ou équivalent). Exemple : feat(api): ajoute l'endpoint POST /users.
- Pull requests / Merge requests obligatoires sur la branche main avec au minimum 1 reviewer.
- Pas de commits avec des secrets, des fichiers volumineux, ou des dossiers node\_modules / vendor / .env. Un .gitignore propre est obligatoire.
- Au moins 4 contributeurs identifiés dans l'historique Git (un par membre du groupe). Un dépôt avec 95 % des commits par une seule personne est sanctionné.

### 2.3 Répartition du travail dans le groupe

- Chaque membre du groupe doit avoir une responsabilité principale identifiable (ex. : lead BDD, lead back-end, lead front-end, lead conception/UX), sans pour autant travailler en silo.
- Un tableau de répartition figure obligatoirement dans le dossier (qui a fait quoi, en heures estimées et en livrables produits).
- Un seul nom dominant sur le dépôt = sanction sur la note de groupe et possibilité de notes différenciées.

## Partie 3 : Dossier de conception (sommaire imposé)

**Le dossier de conception est le livrable principal.** Il sera lu en intégralité par le correcteur et utilisé comme support de la soutenance. Format : PDF unique, en français, propre, paginé, avec page de garde, table des matières et table des figures auto-générées.

**Le sommaire ci-dessous est imposé :** vous devez le suivre dans l'ordre, intégrer toutes les sections, et numéroté chaque section pour faciliter la lecture. Une section absente est une section à zéro.

### Sommaire imposé du dossier

| §   | Section                              | Contenu attendu                                                                                                                                                                  |
|-----|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.  | Page de garde                        | Nom du projet, logo, noms des 4 membres, classe, année, enseignant, date de rendu.                                                                                               |
| 2.  | Table des matières                   | Auto-générée par votre traitement de texte, à jour.                                                                                                                              |
| 3.  | Table des figures                    | Auto-générée, toutes les figures, tableaux et schémas numérotés et légendés.                                                                                                     |
| 4.  | Introduction / contexte              | Pourquoi ce projet, à qui il s'adresse, dans quel contexte d'usage.                                                                                                              |
| 5.  | Analyse du besoin                    | 5.1 Comment je vais y répondre • 5.2 Récapitulatif du besoin                                                                                                                     |
| 6.  | Stratégie                            | 6.1 Technos • 6.2 User Stories • 6.3 Organisation de travail (heures + répartition) • 6.4 Éco-conception • 6.5 Sécurité • 6.6 Déploiement • 6.7 Outils utilisés                  |
| 7.  | Diagrammes — UML                     | 7.1 Cas d'utilisation • 7.2 Classes • 7.3 Pour CHAQUE cas d'utilisation : diag d'objets pré-séquence + séquence + diag d'objets post-séquence                                    |
| 8.  | Diagrammes — UI/UX                   | 8.1 Zoning • 8.2 Wireframes • 8.3 Maquettes haute-fidélité                                                                                                                       |
| 9.  | Diagrammes — Organisation de travail | Gantt, board kanban, capture des sprints, répartition des rôles.                                                                                                                 |
| 10. | Analyse technique                    | 10.1 Bibliothèques (lesquelles, où, pourquoi) • 10.2 Technologies retenues et justification • 10.3 Contraintes (style, responsive, charte) • 10.4 Liste fonctionnelle exhaustive |
| 11. | Conclusion                           | 11.1 Ce qui a été fait — Recette exhaustive (bien passé / pas bien passé) • 11.2 Perspectives d'évolution • 11.3 Idées d'ajouts • 11.4 Ouverture                                 |

### 3.1 Précisions sur les sections critiques

#### Section 5 — Analyse du besoin



- Personas (2 minimum) : qui sont les utilisateurs cibles, leurs objectifs, leurs frustrations actuelles.
- Story map ou MoSCoW (Must / Should / Could / Won't) priorisant les fonctionnalités.
- Contraintes du commanditaire (vous-mêmes en l'occurrence) explicitées.

## Section 6 — Stratégie

- 6.2 User Stories : minimum 15 user stories au format « En tant que [rôle], je veux [action] afin de [bénéfice] », avec critères d'acceptation.
- 6.3 Organisation : tableau heures par membre × tâche, total cumulé, semaine après semaine.
- 6.4 Éco-conception : choix concrets ayant un impact (taille des images, lazy-loading, requêtes SQL optimisées, cache, etc.). Les déclarations vagues type « on a fait attention » ne comptent pas.
- 6.5 Sécurité : hash des mots de passe (bcrypt/Argon2), gestion des secrets, validation des entrées, protection des injections (SQL et NoSQL — il y a aussi des injections NoSQL), CORS, headers de sécurité.
- 6.6 Déploiement : où l'application tourne, comment elle est démarrée, schéma de la stack docker-compose.

## Section 7 — Diagrammes

### Exigence forte sur les diagrammes

Vous devez créer des diagrammes dans le cadre de vos compétences (BDD, Architecture, Docker... Même si vos diagrammes/logigrammes ne répondent pas exactement à une convention comme Merise ou UML

## Section 10 — Analyse technique

- 10.4 Liste fonctionnelle exhaustive : numérotée, avec ID, intitulé, description courte, BDD impactées (SQL/NoSQL/les deux), endpoint(s) associé(s), statut (fait / partiellement fait / non fait).

## Section 11 — Conclusion

- Recette exhaustive : tableau de tests fonctionnels manuels, scénario par scénario, avec résultat attendu vs résultat obtenu.
- Honnêteté valorisée : un « pas bien passé » bien analysé vaut plus qu'un « tout s'est bien passé » de façade.

## Partie 4 : Livrables attendus

Tous les livrables ci-dessous sont obligatoires. Un livrable absent fait perdre la totalité des points associés et peut entraîner un zéro éliminatoire si c'est le code source ou le dossier.

### 4.1 Tableau récapitulatif des livrables

| ID | Livrable                        | Format                            | Contenu attendu                                                                                                                                                            |
|----|---------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L1 | Dépôt Git public                | .zip ou lien                      | Code source complet : back, front, scripts BDD (SQL d'init + NoSQL d'init), Dockerfiles, docker-compose, README, .env.example, .gitignore.                                 |
| L2 | Dossier de conception           | PDF                               | Suivant strictement le sommaire imposé (Partie 3). 30 à 60 pages indicatif. Page de garde, ToC, ToF auto-générées.                                                         |
| L3 | Scripts BDD                     | Fichiers .sql + .js/.cypher/.json | Création du schéma SQL, jeu de données de démo, scripts d'init de la BDD NoSQL, requêtes types commentées.                                                                 |
| L4 | Dossier de captures et extraits | PDF ou intégré au L2              | Screenshots de l'application, extraits de code commentés, requêtes SQL/NoSQL exécutées avec résultat, captures du board de gestion de projet, captures de Postman/Swagger. |
| L5 | Charte graphique                | PDF ou intégré au L2              | Logo, palette, typographie, exemples d'application sur l'UI.                                                                                                               |
| L6 | Démo fonctionnelle              | Vidéo OU live à la soutenance     | Application réellement fonctionnelle. Vidéo de secours (3 min max) recommandée en cas de panne le jour J.                                                                  |
| L7 | Soutenance orale                | Présentiel                        | 10 à 15 min de présentation + 5 à 10 min de questions. Tous les membres doivent parler.                                                                                    |

### 4.2 Précisions sur certains livrables

#### L1 — Dépôt Git

- Structure proposée minimale : /backend, /frontend, /db (sql + nosql), /docs, /docker, README.md à la racine.
- README.md détaillé : présentation du projet, prérequis, instructions de lancement (docker-compose up), variables d'environnement, comptes de démo, lien vers la doc API et le dossier.
- Branches : main (protégée) + une branche par feature (au moins 8 branches sur la durée du projet).

#### L4 — Captures et extraits exigés

##### Liste minimale des captures attendues dans le dossier

- Vue de l'application (page d'accueil, page connectée, formulaires principaux) — 4 captures minimum
- Vue de la documentation API (Swagger/Playground) avec endpoints visibles
- Tests d'API : Postman / Insomnia / Bruno ou cURL — au moins 6 captures (succès + erreurs 4xx)
- Vue du SGBD SQL (DBeaver, MySQL Workbench, pgAdmin) avec les tables peuplées
- Vue du SGBD NoSQL (MongoDB Compass, Redis CLI, Neo4j Browser...) avec les données
- Au moins 3 requêtes SQL complexes exécutées avec leur résultat
- Au moins 3 requêtes NoSQL exécutées avec leur résultat
- Le board Kanban / Scrum avec les tâches en cours et terminées
- L'historique Git (graph) montrant les branches et merges
- La pipeline CI si vous en avez mise une en place (bonus)

## L6 — Démo

- Idéalement live, en local sur l'ordinateur du groupe, avec docker-compose.
- Vidéo de secours fortement recommandée (3 min max, MP4, < 100 Mo, fournie en lien).
- Comptes de démo (admin + utilisateur standard) prêts à l'emploi.

## L7 — Soutenance

- 10 à 15 minutes de présentation, 5 à 10 minutes de questions.
- Tous les membres prennent la parole. Un membre silencieux est un membre noté à zéro sur la soutenance.
- Plan de présentation suggéré : contexte → choix techniques → MCD/schéma NoSQL → démo live → retour d'expérience.
- Support de présentation (diapo) OBLIGATOIRE
- Le jury posera des questions techniques précises sur la BDD : « pourquoi ce type de NoSQL ? », « comment se passe une transaction côté SQL ? », « combien d'index, et pourquoi ceux-là ? », « où est la cohérence entre les deux bases ? ». Préparez-vous.

## Partie 5 : Barème détaillé (sur 20 points)

### Logique de pondération

Le projet est centré BDD : la conception relationnelle, le SQL avancé et le NoSQL pèsent à eux seuls 10 points sur 20. Le développement applicatif, le dossier, la gestion de projet et l'oral se partagent les 10 points restants. Une application fonctionnelle mais sans profondeur côté BDD ne peut pas dépasser 10/20.

| # | Critère                               | / pts     | Détail                                                                                                                                                                                                                                   |
|---|---------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Conception BDD relationnelle</b>   | <b>5</b>  | MCD complet et normalisé (3NF), MLD cohérent, cardinalités correctes, justification des choix de modélisation, gestion propre des associations n-aires et héritages.                                                                     |
| 2 | <b>Maîtrise SQL avancé</b>            | <b>3</b>  | Schéma cohérent et normalisé, vue + trigger + procédure stockée + transaction + index utilisés à bon escient, requête complexe avec jointures multiples + agrégation, scripts d'init reproductibles.                                     |
| 3 | <b>Choix et exploitation du NoSQL</b> | <b>5</b>  | Justification rigoureuse du type retenu (vs les 3 autres types), schéma NoSQL réfléchi, exploitation effective des spécificités (agrégation, TTL, index spécialisés...), au moins une opération métier qui repose principalement dessus. |
| 4 | <b>Développement applicatif</b>       | <b>2</b>  | 16 endpoints REST OU GraphQL équivalent, codes HTTP corrects, doc API auto-générée, séparation back/front, application réellement utilisable.                                                                                            |
| 5 | <b>Dossier de conception</b>          | <b>2</b>  | Sommaire imposé respecté à la lettre, qualité des diagrammes UML (les 9 diagrammes de séquence/objets exigés), profondeur de l'analyse technique, propreté rédactionnelle.                                                               |
| 6 | <b>Gestion de projet &amp; Git</b>    | <b>1</b>  | Board agile actif, répartition équilibrée, commits propres, branches utilisées, MR/PR avec review, .gitignore propre.                                                                                                                    |
| 7 | <b>Soutenance orale &amp; démo</b>    | <b>2</b>  | Démo qui marche, prise de parole équilibrée, capacité à répondre aux questions techniques sur la BDD, qualité de la présentation visuelle.                                                                                               |
|   | <b>TOTAL</b>                          | <b>20</b> | Note finale du projet (commune au groupe sauf cas avéré de déséquilibre)                                                                                                                                                                 |

### 5.1 Pénalités et bonus

#### Pénalités possibles (cumulables)

- Orthographe / lisibilité du dossier : jusqu'à -2 pts.
- Livrable manquant ou incomplet : -1 à -3 pts selon le livrable.

- Contributions Git très déséquilibrées (un seul contributeur dominant) : -1 à -3 pts sur les concernés.
- Plagiat avéré (entre groupes ou depuis Internet) : note éliminatoire (0/20).
- Usage massif d'IA non déclaré : Points perdus sur chaque point non déclaré (peut coûter très cher au final).

**Bonus possibles (jusqu'à +2 pts cumulés)**

- CI/CD fonctionnelle (GitLab CI, GitHub Actions) : lint + tests + build : +0,5 pt.
- Tests automatisés : couverture > 50% sur le back-end : +0,5 pt.
- Déploiement réel et accessible publiquement (avec lien fonctionnel à la soutenance) : +0,5 pt.
- Utilisation pertinente de fonctions avancées non vues en cours (sharding, replication SQL, CQRS, event sourcing, full-text search avancé) : jusqu'à +1 pt.
- Identité graphique particulièrement réussie et originale : +0,5 pt.

## Partie 6 : Calendrier indicatif et modalités de rendu

### 6.1 Découpage indicatif des 10 heures

| Phase          | Thème                                 | Activités attendues                                                                                                                                   |
|----------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Séance 1 (~2h) | Cadrage & conception                  | Choix du sujet, personas, user stories, premier MCD, choix du type NoSQL, choix techniques. Ouverture du dépôt Git et du board.                       |
| Séance 2 (~2h) | Modélisation & maquettes              | Finalisation MCD/MLD, schéma NoSQL, diagrammes UML (cas d'utilisation, classes), zoning et wireframes. Scripts d'init SQL.                            |
| Séance 3 (~2h) | Développement back-end                | API : modèles, routes, contrôleurs, premiers endpoints CRUD. Connexion aux deux BDD. Premiers tests via Postman/Swagger.                              |
| Séance 4 (~2h) | Développement front-end & intégration | UI selon la charte graphique, intégration avec l'API, premiers écrans complets. Vues, triggers, procédures stockées côté SQL. Agrégations côté NoSQL. |
| Séance 5 (~2h) | Finalisation, dossier, soutenance     | Diagrammes de séquence et d'objets, recette, captures, mise au propre du dossier. Préparation de la soutenance.                                       |
| Hors séances   | Travail personnel                     | Rédaction du dossier, écriture des derniers tests, peaufinage de la charte graphique, répétition de la soutenance.                                    |

### 6.2 Modalités de rendu

- Date de rendu des livrables L1 à L6 : à préciser par l'enseignant (généralement la veille de la soutenance, 23h59).
- Soutenances : créneaux communiqués au moins une semaine à l'avance.
- Mode de dépôt : un mail unique par groupe contenant : lien Git, dossier PDF en pièce jointe (ou lien si > 25 Mo), lien vers la vidéo de secours, lien vers le board agile.
- Tout retard est sanctionné : -2 pts par jour de retard, jusqu'à zéro au bout de 3 jours.
- Aucun rendu accepté après le passage à l'oral : pas de soutenance sans dossier déposé.

### 6.3 Composition du jury

- L'enseignant titulaire (Milo Roche).
- Un second évaluateur possible (autre enseignant ou intervenant) selon disponibilités.
- Les autres groupes peuvent assister aux soutenances avec accord du jury.

## Conseils pratiques et pièges fréquents

## À FAIRE

- **Commencez par le MCD.** Une BDD bien modélisée vous fera gagner des heures sur le dev. Une BDD mal modélisée vous bloquera à mi-parcours.
- **Justifiez le choix NoSQL avant de coder.** Si vous ne savez pas pourquoi vous prenez MongoDB plutôt que Redis ou Neo4j, vous prendrez une mauvaise décision.
- **Faites un walking skeleton dès la séance 3 :** 1 endpoint qui lit les 2 BDD et qu'un front affiche. Ensuite, vous étoffez. Trop de groupes échouent à intégrer back+front+SQL+NoSQL en fin de projet.
- **Utilisez Docker dès le départ.** Vous éviterez le syndrome « ça marche chez moi » et vous rendrez la démo de soutenance triviale.
- **Documentez au fil de l'eau,** pas la veille du rendu. Capturez vos écrans pendant que vous codez, pas en revenant en arrière.

## À NE PAS FAIRE

- **Prendre MongoDB par défaut sans réflexion.** Si vos données sont fortement relationnelles, vous allez réinventer une BDD relationnelle dans MongoDB et le regretter.
- **Stocker des passwords en clair, des tokens dans le code, des clés API dans le dépôt Git.** Le correcteur regarde le dépôt en détail.
- **Faire 16 endpoints qui sont tous des SELECT \* basiques.** Le sujet exige de la profondeur, pas du remplissage.
- **Sous-traiter le dossier à l'IA.** Les correcteurs reconnaissent un dossier généré : structure trop régulière, tirets cadratins partout, contenu générique sans ancrage dans votre projet réel. Ce sera détecté en soutenance.
- **Travailler tous sur la branche main sans review.** Vous vous retrouverez avec des conflits Git ingérables et un historique illisible.

**Bon courage à toutes et tous. Pensez à relire votre dossier avant de le rendre, et à tester votre démo sur un autre poste que le vôtre.**